RENTFLOW

Intelligent Property & Tenant Management Portal

Agent Build Specification — v1.0

Framework	Blazor WebAssembly (.NET 8)
Course	Visual Programming (CS)
Institution	Faculty of Computing & Artificial Intelligence, Islamabad
Team Size	3 Students
Deadline	17 May 2026
Document Type	AI Agent Implementation Guide

This document is the single source of truth for AI agents building RentFlow. Follow every section in order. All instructions are self-contained.

Table of Contents

1. Executive Summary
2. Problem Statement
3. User Roles & Permissions
   - 3.1 Super-Admin
   - 3.2 Landlord
   - 3.3 Tenant (Rent-Payer)
4. Tech Stack & Tooling
5. Project File Structure
6. Database Schema
   - 6.1 Entity Definitions
   - 6.2 Relationships Diagram (textual)
   - 6.3 Mock Seed Data
7. Authentication & Authorization Flow
8. UI/UX Design System
   - 8.1 Color Palette & Typography
   - 8.2 Layout & Component Guidelines
9. Feature Specifications by Role
   - 9.1 Admin Dashboard
   - 9.2 Landlord Dashboard
   - 9.3 Tenant Dashboard
10. API Integration Guide
    - 10.1 OpenWeatherMap API
    - 10.2 Groq API — Smart Maintenance Bot
    - 10.3 Google Maps / Geolocation API
11. AI Usage Blueprint
12. Page-by-Page UI Specifications
13. Auto-Late Fee Background Service
14. Course Requirements Compliance Matrix
15. Agent Execution Checklist

1. Executive Summary

RentFlow is a fully automated, role-based web application built with Blazor WebAssembly (.NET 8) that digitises and intelligentises the relationship between landlords and tenants. It replaces fragmented, manual property management workflows with a centralised, API-driven ecosystem covering rent tracking, weather-aware property protection, AI-powered maintenance triage, and automated financial penalty enforcement.

Three Intelligent Core Automations

- Automated Weather Alerts — queries OpenWeatherMap, pushes in-app toast + email when freezing temps or storms detected.
- Smart Maintenance Bot — Groq-powered NLP chatbot that categorises tickets, requests photos, and routes to the right tradesperson.
- Auto-Late Fee Engine — C# hosted background service runs on the 5th of each month and applies 5% late fees to overdue rent.

2. Problem Statement

Maintenance Delays	Tenants submit vague, unstructured repair requests leading to slow routing and escalating property damage.
Environmental Risks	Properties suffer avoidable damage (burst pipes, storm damage) because landlords and tenants receive no proactive weather intelligence.
Payment Tracking Overhead	Landlords manually calculate late fees and reconcile rent payments — a time-intensive and error-prone process.
Communication Fragmentation	Tenants, landlords, and tradespeople communicate across email, phone, and paper — with no single audit trail.

3. User Roles & Permissions

RentFlow has three distinct roles. Each role has a dedicated dashboard, navigation, and data visibility scope. Role is stored in the Users table and enforced via ASP.NET authorization attributes on all API endpoints and Blazor route guards.

3.1 Super-Admin

The Super-Admin is the platform operator. There is exactly one seeded Admin account. The Admin does NOT manage physical properties — that is the Landlord's job. The Admin oversees the entire user base and platform health.

Permission	Details
View all landlords	List, search, suspend, or delete landlord accounts
View all tenants	List, search, suspend, or delete tenant accounts
View all properties	Read-only view of every registered property
View all maintenance tickets	Global audit across all properties
View platform analytics	Aggregated revenue, ticket volumes, active leases
Manage system settings	Configure late fee %, fee trigger day, alert thresholds
Seed / reset demo data	Button to re-seed mock data in development mode

3.2 Landlord

Landlords register themselves and then add their properties and units. They invite tenants by email. All financial and maintenance data flows through their dashboard.

Permission	Details
Add/Edit/Delete properties	CRUD on Property and Unit entities they own
Add/Remove tenants	Assign tenants to units; generate lease records
View rent status	See paid / pending / late payments across all units
View maintenance tickets	See all tickets raised against their properties
Update ticket status	Mark tickets In Progress or Resolved; assign tradesperson
View revenue chart	Chart.js bar chart of monthly income vs expenses
Receive weather alerts	Toast + email when adverse weather hits their property location
View late fee log	Audit trail of auto-applied late fees

3.3 Tenant (Rent-Payer)

Tenants are invited by a Landlord via email. They self-register using the invite link and are automatically linked to their unit and lease. The tenant experience is mobile-first and conversational.

Permission	Details
View dashboard	Current rent balance, due date, weather alert banners
Use Maintenance Bot	Chat with Groq-powered bot; upload photo; submit ticket
View ticket history	List of all submitted tickets and their current status
View payment history	Table of all past payments including late fees
Receive weather alerts	In-app toast when property location has severe weather
View unit info	Unit number, address, landlord contact, lease dates

4. Tech Stack & Tooling

Layer	Technology	Version / Notes
Frontend Framework	Blazor WebAssembly	.NET 8 — component-based SPA
UI Styling	Bootstrap 5 + Custom CSS	Custom CSS variables map to design tokens
Charts	Chart.js via JS Interop	v4.x — CDN import in index.html
Backend / API Host	ASP.NET Core Web API	.NET 8 — hosts the WASM and REST endpoints
ORM	Entity Framework Core	v8.x with SQLite provider
Database	SQLite	Single-file DB; rentflow.db in /data/ folder
Auth	ASP.NET Core Identity + JWT	JWT tokens stored in Blazor LocalStorage
State Management	Blazored.LocalStorage	NuGet: Blazored.LocalStorage 4.x
Chatbot / NLP	Groq API	Model: llama3-8b-8192; HttpClient call from server
Weather	OpenWeatherMap API	Current Weather endpoint; free tier sufficient
Geolocation	Google Maps JS API	JS Interop: getCurrentPosition()
Background Jobs	IHostedService (C#)	LateFeeService runs daily; checks day-of-month
File Uploads	Blazor InputFile Component	Saved to /uploads/ folder on server
Toast Notifications	Custom Blazor Service	ToastService injected app-wide
Email Alerts	SMTP (MailKit)	NuGet: MailKit; configure in appsettings.json
Version Control	Git	GitHub repo; main + feature branches
IDE	Visual Studio 2022 / VS Code	VS2022 recommended; Rider also works
Package Manager	NuGet + npm (for Chart.js)	npm only for Chart.js if not using CDN

NuGet Packages to Install

```
Microsoft.EntityFrameworkCore.Sqlite
Microsoft.EntityFrameworkCore.Tools
Microsoft.AspNetCore.Identity.EntityFrameworkCore
Microsoft.AspNetCore.Authentication.JwtBearer
Blazored.LocalStorage
MailKit
Microsoft.AspNetCore.Components.WebAssembly (client project)
Microsoft.AspNetCore.Components.WebAssembly.Server (server project)
```

5. Project File Structure

Use the Blazor WebAssembly Hosted template which creates three projects in one solution: Client (WASM), Server (ASP.NET Core), and Shared (DTOs + models). Command: dotnet new blazorwasm --hosted -n RentFlow

```
RentFlow/ # Solution root
■■■ RentFlow.sln
■■■ RentFlow.Client/ # Blazor WASM frontend
■ ■■■ Pages/
■ ■ ■■■ Auth/
■ ■ ■ ■■■ Login.razor
■ ■ ■ ■■■ Register.razor
■ ■ ■■■ Admin/
■ ■ ■ ■■■ AdminDashboard.razor # Platform analytics
■ ■ ■ ■■■ ManageLandlords.razor # CRUD on landlord accounts
■ ■ ■ ■■■ ManageTenants.razor # CRUD on tenant accounts
■ ■ ■■■ Landlord/
■ ■ ■ ■■■ LandlordDashboard.razor
■ ■ ■ ■■■ Properties.razor # List all properties
■ ■ ■ ■■■ PropertyDetail.razor # Units + map embed
■ ■ ■ ■■■ AddEditProperty.razor # Create/edit form
■ ■ ■ ■■■ Tenants.razor # Manage tenant assignments
■ ■ ■ ■■■ MaintenanceRequests.razor
■ ■ ■ ■■■ RevenueChart.razor # Chart.js dashboard
■ ■ ■■■ Tenant/
■ ■ ■ ■■■ TenantDashboard.razor # Rent status + alerts
■ ■ ■ ■■■ MaintenanceBot.razor # Groq chat interface
■ ■ ■ ■■■ PaymentHistory.razor
■ ■ ■ ■■■ MyUnit.razor # Lease + unit info
■ ■ ■■■ Index.razor # Public landing/redirect
■ ■■■ Shared/
■ ■ ■■■ MainLayout.razor
■ ■ ■■■ AdminLayout.razor / LandlordLayout.razor / TenantLayout.razor
■ ■ ■■■ NavMenu.razor
■ ■ ■■■ ToastContainer.razor # Global toast host
■ ■ ■■■ WeatherAlertBanner.razor # Full-width alert strip
■ ■ ■■■ LoadingSpinner.razor
```

```
■ ■■■ Services/ # Client-side service layer
■ ■ ■■■ AuthService.cs # JWT store + login calls
■ ■ ■■■ PropertyService.cs / MaintenanceService.cs / PaymentService.cs
■ ■ ■■■ WeatherService.cs # Calls /api/weather on server
■ ■ ■■■ GeolocationService.cs # JS Interop wrapper
■ ■ ■■■ ToastService.cs # In-app notification event bus
■ ■■■ Helpers/RouteGuard.cs # Role-based redirect helper
■ ■■■ wwwroot/
■ ■ ■■■ css/app.css # All custom styles
■ ■ ■■■ css/variables.css # CSS custom properties (design tokens)
■ ■ ■■■ js/interop.js # JS Interop: geolocation, etc.
■ ■ ■■■ js/charts.js # Chart.js wrapper functions
■ ■ ■■■ index.html # CDN imports: Bootstrap, Chart.js, Inter
■ ■■■ Program.cs / _Imports.razor
■■■ RentFlow.Server/ # ASP.NET Core API + WASM host
■ ■■■ Controllers/
■ ■ ■■■ AuthController.cs # POST /api/auth/login|register
■ ■ ■■■ PropertyController.cs # CRUD /api/properties
■ ■ ■■■ MaintenanceController.cs # CRUD + file upload
■ ■ ■■■ PaymentController.cs # Payments + late fee query
■ ■ ■■■ WeatherController.cs # Proxies OpenWeatherMap
■ ■ ■■■ ChatController.cs # Proxies Groq API
■ ■ ■■■ AdminController.cs # Platform-wide queries
■ ■■■ Data/AppDbContext.cs / SeedData.cs
■ ■■■ Services/
■ ■ ■■■ LateFeeBackgroundService.cs
■ ■ ■■■ EmailService.cs
■ ■ ■■■ TokenService.cs
■ ■■■ Migrations/ / data/rentflow.db / uploads/
■ ■■■ appsettings.json # ALL API keys stored here
■■■ RentFlow.Shared/ # Shared models & DTOs
■■■ Models/ (User, Property, Unit, Lease, Payment, MaintenanceTicket, WeatherAlertLog)
■■■ DTOs/ (LoginDto, RegisterDto, PropertyDto, MaintenanceTicketDto, PaymentDto)
```

6. Database Schema

All entities live in a single SQLite database (rentflow.db). EF Core manages migrations. Run 'dotnet ef migrations add InitialCreate' and 'dotnet ef database update' from the Server project. The SeedData.cs class is called from Program.cs on startup and is idempotent (checks before inserting).

Table: Users

Column	Type	Constraints	Description
Id	int	PK, IDENTITY	Auto-increment primary key
FullName	string	NOT NULL	Display name
Email	string	UNIQUE, NOT NULL	Used as login username
PasswordHash	string	NOT NULL	BCrypt hash via ASP.NET Identity
Role	string	NOT NULL	Admin | Landlord | Tenant
IsActive	bool	DEFAULT true	Admin can deactivate
CreatedAt	DateTime	DEFAULT NOW	UTC timestamp

Table: Properties

Column	Type	Constraints	Description
Id	int	PK, IDENTITY	
LandlordId	int	FK -> Users.Id	Owner of this property
Name	string	NOT NULL	e.g., 'Sunset Apartments'
Address	string	NOT NULL	Full street address
City	string	NOT NULL	
Latitude	double	NULLABLE	From Geolocation API
Longitude	double	NULLABLE	From Geolocation API
TotalUnits	int	NOT NULL	Number of rentable units
CreatedAt	DateTime	DEFAULT NOW	

Table: Units

Column	Type	Constraints	Description
Id	int	PK, IDENTITY	
PropertyId	int	FK -> Properties.Id	Parent property
UnitNumber	string	NOT NULL	e.g., '101', 'A2'
MonthlyRent	decimal	NOT NULL	Base rent in PKR
Bedrooms	int	DEFAULT 1	
IsOccupied	bool	DEFAULT false	Updated on lease creation

Table: Leases

Column	Type	Constraints	Description
Id	int	PK, IDENTITY	
UnitId	int	FK -> Units.Id	
TenantId	int	FK -> Users.Id	Must have Role=Tenant
StartDate	DateTime	NOT NULL	
EndDate	DateTime	NULLABLE	Null = open-ended
MonthlyRent	decimal	NOT NULL	Agreed rent amount
IsActive	bool	DEFAULT true	

Table: Payments

Column	Type	Constraints	Description
Id	int	PK, IDENTITY	
LeaseId	int	FK -> Leases.Id	
TenantId	int	FK -> Users.Id	Denormalised for fast query
DueDate	DateTime	NOT NULL	1st of each month
PaidDate	DateTime	NULLABLE	Null = unpaid
Amount	decimal	NOT NULL	Base rent
LateFee	decimal	DEFAULT 0	Applied by background service
Status	string	NOT NULL	Pending | Paid | Late

Table: MaintenanceTickets

Column	Type	Constraints	Description
Id	int	PK, IDENTITY	
TenantId	int	FK -> Users.Id	Raised by
PropertyId	int	FK -> Properties.Id	
UnitId	int	FK -> Units.Id	
Category	string	NOT NULL	Plumbing|Electrical|HVAC|Structural|Other
Description	string	NOT NULL	Natural language from bot
PhotoPath	string	NULLABLE	Server path to uploaded image
Status	string	NOT NULL	Open | InProgress | Resolved
AssignedTo	string	NULLABLE	Tradesperson name/contact
BotTranscript	string	NULLABLE	JSON of chat messages
CreatedAt	DateTime	DEFAULT NOW	
UpdatedAt	DateTime	DEFAULT NOW	Updated on status change

Table: WeatherAlertLogs

Column	Type	Constraints	Description
Id	int	PK, IDENTITY	
PropertyId	int	FK -> Properties.Id	Affected property
AlertType	string	NOT NULL	Freeze | Storm | Heatwave | Flood
Message	string	NOT NULL	Human-readable instruction
TriggeredAt	DateTime	DEFAULT NOW	
IsRead	bool	DEFAULT false	Cleared by tenant/landlord

6.2 Entity Relationships

```
Users (1) ■■■■■■< Properties (M) [via LandlordId]
Properties (1) ■■■■■< Units (M) [via PropertyId]
Units (1) ■■■■■■< Leases (M) [one active lease per unit]
Leases (1) ■■■■■< Payments (M) [one per month]
Users (1) ■■■■■■< Leases (M) [tenant side]
Users (1) ■■■■■■< MaintenanceTickets [via TenantId]
Properties (1) ■< MaintenanceTickets [via PropertyId]
Properties (1) ■< WeatherAlertLogs [via PropertyId]
```

6.3 Mock Seed Data — SeedData.cs

The following data must be seeded on first run via SeedData.cs. All passwords are 'RentFlow@2024' hashed with BCrypt. The seeder checks if any users exist before inserting (idempotent).

Account	Email	Role	Password
Platform Admin	admin@rentflow.io	Admin	RentFlow@2024
Ahmed Raza	ahmed.landlord@rentflow.io	Landlord	RentFlow@2024
Sara Malik	sara.landlord@rentflow.io	Landlord	RentFlow@2024
Ali Hassan	ali.tenant@rentflow.io	Tenant	RentFlow@2024
Fatima Noor	fatima.tenant@rentflow.io	Tenant	RentFlow@2024
Bilal Khan	bilal.tenant@rentflow.io	Tenant	RentFlow@2024
Zara Siddiqui	zara.tenant@rentflow.io	Tenant	RentFlow@2024

Property	Address	City	Landlord	Units
Gulberg Heights	12-B, Gulberg III	Lahore	Ahmed Raza	4
Blue Area Plaza	Plot 7, Blue Area	Islamabad	Ahmed Raza	3
Clifton Residency	House 22, Clifton Block 4	Karachi	Sara Malik	2

Seed 6 units across the above properties. Assign each of the 4 tenants to a unit with an active lease (start Jan 2024). Create 4 months of Payment records per tenant: 3 with status Paid, 1 with status Late (LateFee = MonthlyRent * 0.05). Create 4 MaintenanceTickets: 2 Open, 1 InProgress, 1 Resolved. Create 1 WeatherAlertLog per property.

7. Authentication & Authorization Flow

1. User visits /login	Public route. No auth required. Redirect authenticated users to their role dashboard.
2. Submit credentials	POST /api/auth/login with { email, password }. Server validates via ASP.NET Identity.
3. Server returns JWT	JWT payload includes: userId, email, role, fullName, exp (24h). Return 401 on fail.
4. Client stores JWT	Store in Blazored.LocalStorage under key 'rentflow_token'. NEVER in sessionStorage.
5. AuthService parses JWT	Decode payload (no library needed — base64 split). Set ClaimsPrincipal. Notify AuthStateProvider.
6. Role-based redirect	Role == Admin -> /admin/dashboard. Landlord -> /landlord/dashboard. Tenant -> /tenant/dashboard.
7. API calls	Every HttpClient call adds header: Authorization: Bearer {token}. Use a DelegatingHandler.
8. Route guards	Add [Authorize(Roles="Landlord")] on Razor pages. Client-side: redirect in OnInitializedAsync.
9. Logout	Clear LocalStorage token. Call AuthStateProvider.NotifyUserLoggedOut(). Redirect to /login.
10. Registration	POST /api/auth/register with { fullName, email, password, role }. Role MUST be Landlord or Tenant (Admin is seeded only).

Security Rules

- Never store plaintext passwords — use ASP.NET Identity's PasswordHasher.
- JWT secret key in appsettings.json: Jwt:Key (min 32 chars). Never commit to Git.
- All /api/* routes require [Authorize] except /api/auth/login and /api/auth/register.
- CORS: Allow only the WASM client origin in development (http://localhost:5001).

8. UI/UX Design System

8.1 Color Palette & Typography

Token	Hex Value	Usage
--color-navy	#1B2A4A	Primary brand, sidebar background, headings
--color-navy-light	#2E4070	Sidebar hover state, card headers
--color-teal	#0EA5E9	Primary actions, CTAs, active links, badges
--color-teal-light	#BAE6FD	Info box backgrounds, chip backgrounds
--color-amber	#F59E0B	Weather alert banners, warning states
--color-amber-light	#FEF3C7	Warning backgrounds, late fee highlights
--color-red	#EF4444	Late rent status, error states, delete buttons
--color-red-light	#FEE2E2	Error backgrounds, overdue row highlights
--color-green	#10B981	Paid status, success toasts, resolved tickets
--color-green-light	#D1FAE5	Success backgrounds
--color-white	#FFFFFF	Page backgrounds, card surfaces
--color-gray-100	#F8FAFC	Alternating table rows, input backgrounds
--color-gray-200	#E2E8F0	Borders, dividers, table grid lines
--color-gray-700	#334155	Body text, secondary labels

Typography

Role	Font	Weight	Size
Page Title (H1)	Inter / Helvetica	700	28px
Section Title (H2)	Inter / Helvetica	600	22px
Card Title (H3)	Inter / Helvetica	600	16px
Body Text	Inter / Helvetica	400	14px
Table Text	Inter / Helvetica	400	13px
Badge / Chip	Inter / Helvetica	600	11px
Code / Monospace	Courier New / Fira Code	400	13px

Import Inter from Google Fonts in index.html: https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display;=swap Then set: body {

font-family: 'Inter', sans-serif; } in app.css

8.2 Layout & Component Guidelines

- **Sidebar Navigation**

  Fixed 240px left sidebar. Navy background. Logo at top. Role-specific nav links. Collapse to icon-only at < 768px. Active link highlighted with left teal border.

- **Top Header Bar**

  60px height. White background. Page title left. User avatar + name + logout button right. On Tenant/Landlord views: show weather alert bell icon that opens alert drawer.

- **Dashboard Cards**

  Grid of stat cards (4-across desktop, 2-across tablet, 1-across mobile). Each card: icon top-left, big number, label, colour-coded border-left 4px. Use Bootstrap col-xl-3 col-md-6 col-12.

- **Data Tables**

  Bootstrap table-hover table-bordered. Header row: Navy bg, white text. Alternating rows: white / gray-100. Status columns: coloured badges. Include search input above table and pagination below.

- **Forms**

  White card with 24px padding. Labels above inputs. Bootstrap form-control. Validation errors in red below field. Submit button: teal, full-width on mobile.

- **Modals**

  Use Bootstrap modals for confirmations (delete, assign tradesperson). Overlay: Navy 60% opacity. Modal: white, rounded-3, max-width 540px.

- **Toast Notifications**

  Bottom-right corner stack. Auto-dismiss after 5s. Success: green left border. Warning: amber. Error: red. Info: teal.

- **Weather Alert Banner**

  Full-width strip below top header. Amber bg #FEF3C7. Amber-800 text. Warning icon left. Message centre. Dismiss X right. Persists until dismissed.

- **Chat Interface (Bot)**

  Full-height flex column. Messages scroll container. Bot messages: left-aligned, gray-100 bubble. User messages: right-aligned, teal bubble. File upload zone appears inline when bot requests photo.

- **Charts**

  Use Chart.js Bar chart for revenue. Canvas element with id='revenueChart'. Colors: datasets use teal for income, red for expenses. Responsive: true, maintainAspectRatio: false, height 300px container.

9. Feature Specifications by Role

9.1 Admin Dashboard (/admin/*)

Page / Route	Components & Behaviour
/admin/dashboard	4 stat cards: Total Landlords, Total Tenants, Active Leases, Open Tickets. Below: recent activity log ta
/admin/landlords	Searchable table of all Landlord accounts (Name, Email, Properties count, Status). Actions: View Deta
/admin/tenants	Searchable table of all Tenant accounts (Name, Email, Unit, Landlord, Payment Status). Actions: View
/admin/properties	Read-only table: Property Name, Address, Landlord, Units, Occupancy %. Click row to expand mainte
/admin/settings	Form: Late Fee Percentage (default 5%), Fee Trigger Day (default 5), Weather Check Interval (hours)

9.2 Landlord Dashboard (/landlord/*)

Page / Route	Components & Behaviour
/landlord/dashboard	4 stat cards: Total Properties, Occupied Units, Rent Due This Month, Open Tickets. Revenue Bar Cha
/landlord/properties	Card grid of properties. Each card: photo placeholder, name, address, unit count, occupancy badge. E
/landlord/properties/{id}	Property header with address + embedded Google Map (JS Interop iframe). Units table: Unit#, Tenant
/landlord/properties/add	Form: Property Name, Address, City, Total Units. Geolocation button: 'Use Current Location' -> JS Int
/landlord/tenants	Table: Tenant Name, Email, Unit, Property, Lease Start, Rent, Payment Status. Invite Tenant button: e
/landlord/maintenance	Filterable table: all tickets for their properties. Filters: Status, Category, Property. Click ticket -> modal:
/landlord/revenue	Full-page Chart.js dashboard. Bar chart: monthly income. Line chart overlay: expected vs actual. Sum

9.3 Tenant Dashboard (/tenant/*)

Page / Route	Components & Behaviour
/tenant/dashboard	Hero card: 'Rent Due: PKR X,XXX on [date]' with large teal CTA if unpaid or red if late. Quick stats: Le
/tenant/maintenance	The Smart Maintenance Bot chat interface (see Section 10.2 for full flow). Below chat: My Tickets tabl
/tenant/payments	Payment history table: Month, Due Date, Paid Date, Amount, Late Fee, Status badge. Overdue rows h
/tenant/unit	Card layout: Unit Number, Floor, Bedrooms, Property Name, Full Address. Embedded map showing p

10. API Integration Guide

API Key Storage

- ALL API keys are stored in RentFlow.Server/appsettings.json ONLY. Never in client-side code.
- The client calls your own server endpoints (/api/weather, /api/chat) which then call external APIs.
- Format in appsettings.json: { "ApiKeys": { "OpenWeatherMap": "KEY", "Groq": "KEY", "GoogleMaps": "KEY" } }
- Use IConfiguration injection to read keys in controllers.

10.1 OpenWeatherMap API

Used to fetch current weather for each property's latitude/longitude. A background task polls every 6 hours per property and triggers alerts on threshold breach.

Item	Detail
Endpoint	GET https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={key}&units=metric
Trigger: Freeze	temp < 2°C — Alert type: Freeze. Message: 'Freezing temps detected. Drip faucets and insulate pipes.'
Trigger: Storm	weather[0].main == 'Thunderstorm' — Alert type: Storm. Message: 'Storm warning. Secure outdoor furnitu
Trigger: Heatwave	temp > 42°C — Alert type: Heatwave. Message: 'Extreme heat. Check HVAC units and ventilation.'
Server endpoint	GET /api/weather/{propertyId} — WeatherController fetches OWM, evaluates thresholds, saves log, return
Client call	WeatherService.cs calls GET /api/weather/{propertyId}. Result shown in WeatherAlertBanner.razor.
Polling schedule	WeatherBackgroundService runs every 6h. Can be triggered manually via Admin settings page.
Toast trigger	If new alert detected (not in last 24h log), push toast via ToastService and send email via EmailService.

10.2 Groq API — Smart Maintenance Bot

The chatbot is a multi-turn conversation powered by Groq's llama3-8b-8192 model. The Server acts as a proxy — client sends conversation history to /api/chat, server calls Groq, returns assistant reply. The bot follows a structured script.

Step	Bot Action	Expected User Response
1	Greet: 'Hi! I'm the RentFlow Maintenance Bot. Briefly describe the issue you're experiencing.'	Natural language issue description, e.g. 'The kitchen sink is leaking badly'
2	Groq categorises from description. Bot replies: 'Got it — this looks like a Plumbing issue. Is that correct?'	Yes / No correction
3	Bot: 'Could you rate the urgency? 1 = Minor, 2 = Moderate, 3 = Emergency'	Number 1–3
4	Bot: 'Please upload a photo of the issue so we can assess it remotely.'	File upload (optional — FileUpload component appears inline)
5	Bot confirms: 'Thank you. Your ticket has been submitted and will be reviewed within 24-48 hours.'	Conversation ends. Ticket created in DB.

Groq API Call Specification

```
// ChatController.cs — POST /api/chat
var client = new HttpClient();
client.DefaultRequestHeaders.Add("Authorization", $"Bearer {_config["ApiKeys:Groq"]}");
var body = new {
model = "llama3-8b-8192",
messages = new[] {
new { role = "system", content = SYSTEM_PROMPT },
// ... conversation history from client
},
max_tokens = 300,
temperature = 0.3
};
// POST to: https://api.groq.com/openai/v1/chat/completions
// Parse: response.choices[0].message.content
```

System Prompt for Groq Bot

```
You are RentFlow Maintenance Bot. Your ONLY job is to collect:
1. A description of the maintenance issue
2. Confirm the category (Plumbing/Electrical/HVAC/Structural/Other)
3. Urgency level (1=Minor, 2=Moderate, 3=Emergency)
4. Confirm photo has been uploaded
Then output a JSON summary: {"category":"Plumbing","urgency":2,"description":"..."}
Keep responses SHORT (max 2 sentences). Do not go off-topic.
When all info collected, respond ONLY with the JSON and nothing else.
```

10.3 Google Maps / Geolocation API

Usage	Implementation
Get user's current location	JS Interop call in GeolocationService.cs — navigator.geolocation.getCurrentPosition() returns {latitude
Embed map on property page	Google Maps Embed API iframe: https://www.google.com/maps/embed/v1/place?key={KEY}&q={lat},{
Tenant unit page map	Same embed URL using property's stored lat/lng from DB.
JS Interop function	In interop.js: window.getLocation = () => new Promise((res,rej) => navigator.geolocation.getCurrentPo
C# call	await JSRuntime.InvokeAsync<LocationDto>("getLocation"); — catches GeolocationPositionError.
API Key restriction	Restrict Google API key to Maps Embed API and Geolocation API only in Google Cloud Console.

11. AI Usage Blueprint

This section maps every location in the application where AI / external intelligence is used, so agents know exactly what to build vs what to call.

Feature	Where in Code	AI/Service Used	Input	Output
Maintenance Categorisation	ChatController.cs MaintenanceBot.razor	Groq llama3-8b	User's natural language description + conversation history	Category string + confirmation message
Ticket Urgency Assessment	ChatController.cs	Groq llama3-8b	Description + category	Urgency level 1-3
Final Ticket JSON Extraction	ChatController.cs — parseTicketFromResponse()	Groq llama3-8b	Full conversation	Structured JSON: category, urgency,
Weather Threshold Alert	WeatherBackgroundService.cs WeatherController.cs	OpenWeatherMap API	Property lat/lng	Current temp, weather condition code
Alert Message Generation	WeatherController.cs (hardcoded rules)	Rule engine (no AI)	Weather condition type	Human-readable instruction string
Property Geolocation	AddEditProperty.razor GeolocationService.cs	Browser Geolocation API	User clicks 'Use My Location'	Lat/lng coordinates saved to Property
Map Rendering	PropertyDetail.razor MyUnit.razor	Google Maps Embed API	Stored lat/lng from DB	Interactive embedded map iframe
Auto-Late Fee Calculation	LateFeeBackgroundService.cs	Pure C# logic (no AI)	Payment records where DueDate passed and Status = Pendi	Updated Payment.LateFee and Status = Overdu

12. Page-by-Page UI Specifications

Login Page (/login)

- Layout: Centred card (480px wide) on navy background. RentFlow logo text top-centre.
- Fields: Email (type=email, placeholder='you@example.com'), Password (type=password). Both required.
- Button: 'Sign In' — full width, teal, 44px height. Shows spinner on submit.
- Error: Red alert box below button for invalid credentials.
- Link: 'Don't have an account? Register here' -> /register.
- On success: JWT stored, redirect to role-appropriate dashboard.
- Do NOT show role selector on this page — role is read from JWT.

Register Page (/register)

- Layout: Same centred card as login.
- Fields: Full Name, Email, Password, Confirm Password, Role (dropdown: Landlord / Tenant only — Admin blocked).
- Validation: Password min 8 chars, must match confirm, email unique check on submit.
- On success: Auto-login, redirect to dashboard based on chosen role.
- Tenants registered this way are NOT linked to a unit — Landlord must assign them.

Admin Dashboard (/admin/dashboard)

- Stat Cards Row: [Total Landlords | teal icon] [Total Tenants | navy icon] [Active Leases | green icon] [Open Tickets | amber icon].
- Platform Activity Table: 10 most recent events (user joined, ticket opened, late fee applied). Columns: Event, User, Time.
- Tickets by Category Chart: Horizontal bar chart. Categories: Plumbing, Electrical, HVAC, Structural, Other.
- Quick Links: Manage Landlords, Manage Tenants, System Settings — large clickable cards in 3-column grid.

Landlord Dashboard (/landlord/dashboard)

- Stat Cards: [My Properties] [Occupied Units / Total Units] [Rent Due This Month (PKR)] [Open Tickets].
- Revenue Chart: Chart.js Bar chart. X-axis: last 6 months. Y-axis: PKR. One bar per month. Teal colour.
- Late Payments Panel: Table of tenants with Status=Late. Columns: Tenant, Unit, Amount, Late Fee, Days Overdue.
- Weather Alerts Panel: List of active WeatherAlertLogs for all owned properties. Amber background. Dismiss button.
- Maintenance Summary: Donut chart — Open vs InProgress vs Resolved tickets.

Tenant Dashboard (/tenant/dashboard)

- Hero Section: Large card. Left: 'Your Rent' label, PKR amount in 32px bold. Right: status badge (Paid=green, Pending=teal, Late=red).
- If Status=Late: Show 'OVERDUE — includes PKR X late fee' in red below amount.
- Weather Alert Banner: Full-width amber strip if any unread alert for property. Shows alert message + dismiss X.
- Quick Info Row: 3 mini cards — [Unit: 101] [Property: Gulberg Heights] [Landlord: Ahmed Raza].
- Recent Tickets: Last 3 tickets as horizontal status chips — Category + Status badge.
- Quick Action: Large 'Report an Issue' button -> /tenant/maintenance.

Maintenance Bot Page (/tenant/maintenance)

- Split layout: LEFT 60% = Chat window. RIGHT 40% = My Tickets table.
- Chat window: Fixed height, overflow-y scroll. Messages rendered in bubbles.
- Bot avatar: Small teal circle with robot emoji. User avatar: initials circle.
- When bot requests photo: InputFile component renders inline as a dashed upload zone in the chat.
- On upload: Show thumbnail preview in chat. File sent to POST /api/maintenance/upload.
- Submit button: Disabled until conversation reaches completion (bot outputs JSON).
- On ticket creation: Show green success toast. Add to My Tickets table without page reload.
- My Tickets table columns: #, Category, Urgency, Status, Created Date. Click row to expand description.

13. Auto-Late Fee Background Service

LateFeeBackgroundService.cs implements IHostedService. It runs as a background task within the ASP.NET Core host. The service checks daily whether it is the 5th of the month, then processes all overdue payments.

```
// LateFeeBackgroundService.cs — key logic
protected override async Task ExecuteAsync(CancellationToken ct) {
while (!ct.IsCancellationRequested) {
await Task.Delay(TimeSpan.FromHours(24), ct);
if (DateTime.UtcNow.Day == 5) { // Configurable via Settings
await ApplyLateFeesAsync();
}
}
}
private async Task ApplyLateFeesAsync() {
var overduePayments = await _db.Payments
.Where(p => p.Status == "Pending" && p.DueDate < DateTime.UtcNow)
.ToListAsync();
foreach (var payment in overduePayments) {
payment.LateFee = payment.Amount * 0.05m; // 5% — configurable
payment.Status = "Late";
// Send email alert to tenant and landlord
await _emailService.SendLateFeeNotificationAsync(payment);
}
await _db.SaveChangesAsync();
}
```

Registration in Program.cs

- builder.Services.AddHostedService();
- builder.Services.AddScoped();
- Register AppDbContext as a factory for use inside the hosted service (scoped lifetime issue):
- builder.Services.AddDbContextFactory(options => options.UseSqlite(...));

14. Course Requirements Compliance Matrix

Requirement	Status	Implementation in RentFlow
Blazor Framework	REQUIRED	Blazor WebAssembly .NET 8 — full component architecture
Real-World Problem	REQUIRED	Property & tenant management — genuine market need
User Authentication	REQUIRED	ASP.NET Identity + JWT; 3 roles with route guards
External API Integration	REQUIRED	OpenWeatherMap + Groq + Google Maps (3 APIs)
Local Data Storage	REQUIRED	SQLite via EF Core — full relational schema
File Uploads	REQUIRED	Blazor InputFile on Maintenance Bot — photos saved server-side
Geolocation (JS Interop)	REQUIRED	navigator.geolocation.getCurrentPosition() via GeolocationService
In-App Notifications	REQUIRED	Custom ToastService + Weather Alert Banner component
Charts / Visualisation	REQUIRED	Chart.js via JS Interop — Revenue bar + Tickets donut
Responsive UI/UX	REQUIRED	Bootstrap 5 grid — tested mobile, tablet, desktop
Component Architecture	REQUIRED	All UI in reusable Razor components; services via DI
State Management	REQUIRED	Blazored.LocalStorage for JWT; cascading auth state
Background Automation	EXTRA	IHostedService for daily late fee engine
Email Alerts	EXTRA	MailKit SMTP integration for weather + late fee emails
Multi-Role Dashboard	EXTRA	3 completely separate dashboard experiences
Mock Data Seeder	EXTRA	SeedData.cs — fully populated demo DB on first run
AI-Powered Bot	EXTRA	Groq LLM for natural language maintenance triage

15. Agent Execution Checklist

Follow these tasks in order. Do not skip ahead. Each task depends on the previous.

PHASE 1 — Project Setup

- ■ Run: dotnet new blazorwasm --hosted -n RentFlow
- ■ Install all NuGet packages listed in Section 4
- ■ Create folder structure exactly as specified in Section 5
- ■ Add CDN links in Client/wwwroot/index.html: Chart.js, Bootstrap 5 CSS+JS, Inter Google Font
- ■ Create CSS custom properties in variables.css from Section 8.1 color table
- ■ Configure appsettings.json with placeholder API keys (agent must not hardcode keys in code)

PHASE 2 — Database & Seed

- ■ Create all model classes in RentFlow.Shared/Models/ per Section 6.1
- ■ Create AppDbContext.cs with DbSet for each entity; configure relationships via Fluent API
- ■ Run: dotnet ef migrations add InitialCreate (from Server project)
- ■ Run: dotnet ef database update
- ■ Create SeedData.cs and call from Program.cs — seed all users, properties, units, leases, payments, tickets per Section 6.3
- ■ Verify: run app, check DB with DB Browser for SQLite — confirm all rows exist

PHASE 3 — Authentication

- ■ Implement AuthController with /login and /register endpoints
- ■ Implement TokenService.cs — generate JWT with userId, email, role claims
- ■ Implement client-side AuthService.cs — login, logout, getUser, isAuthenticated
- ■ Create custom AuthenticationStateProvider that reads JWT from LocalStorage
- ■ Register in Client/Program.cs: builder.Services.AddAuthorizationCore()
- ■ Add [Authorize(Roles=...)] attributes to all role-specific pages
- ■ Test: login as each seeded user and verify dashboard redirect

PHASE 4 — Core CRUD & UI

- ■ Build MainLayout.razor with sidebar + header; create AdminLayout, LandlordLayout, TenantLayout
- ■ Build all Admin pages: AdminDashboard, ManageLandlords, ManageTenants, Properties, Settings
- ■ Build all Landlord pages: LandlordDashboard, Properties, PropertyDetail, AddEditProperty, Tenants, Maintenance, Revenue
- ■ Build all Tenant pages: TenantDashboard, MaintenanceBot, PaymentHistory, MyUnit
- ■ Build Login.razor and Register.razor
- ■ Implement ToastService and ToastContainer.razor
- ■ Implement WeatherAlertBanner.razor

PHASE 5 — API Integrations

- ■ Implement WeatherController.cs — call OWM, evaluate thresholds, save WeatherAlertLog

- ■ Implement client WeatherService.cs — call /api/weather/{propertyId}
- ■ Implement ChatController.cs — proxy conversation to Groq, return response
- ■ Implement MaintenanceBot.razor — full chat UI + file upload + ticket creation on completion
- ■ Implement GeolocationService.cs — JS Interop getCurrentPosition()
- ■ Add JS function window.getLocation in interop.js
- ■ Embed Google Maps iframe in PropertyDetail.razor and MyUnit.razor

PHASE 6 — Charts & Background Services

- ■ Add Chart.js CDN to index.html; create charts.js with renderBarChart() and renderDonutChart() functions
- ■ Add JS Interop calls in RevenueChart.razor and AdminDashboard.razor to invoke chart functions
- ■ Implement LateFeeBackgroundService.cs per Section 13
- ■ Implement EmailService.cs with MailKit for weather + late fee notifications
- ■ Register IHostedService in Program.cs

PHASE 7 — QA & Polish

- ■ Test all user flows: admin CRUD, landlord property add, tenant bot + ticket
- ■ Test weather alert: manually call WeatherController with a test property
- ■ Test late fee: temporarily change condition to check if day >= 1 to trigger service
- ■ Verify mobile responsiveness on all pages (Chrome DevTools 375px width)
- ■ Add loading spinners on all async operations (see LoadingSpinner.razor)
- ■ Final check: all 18 compliance items from Section 14 are met

Final Reminder for Agents

- • API keys will be provided by the student — insert them into appsettings.json ONLY.
- • Do not commit API keys, the rentflow.db file, or the /uploads/ folder to Git.
- • Add .gitignore entries for: *.db, /uploads/, appsettings.json (or use appsettings.Development.json).
- • The app must run with 'dotnet run' from the Server project with zero manual setup beyond API key entry.
- • All mock data must be visible immediately on first launch — no manual DB population required.